

Reliable UDP File Transfer

Go-Back-N Sliding Window 기반 파일 전송 시스템

네트워크 시스템 프로그래밍 기말 프로젝트
개발 환경: Linux , C Language

학번

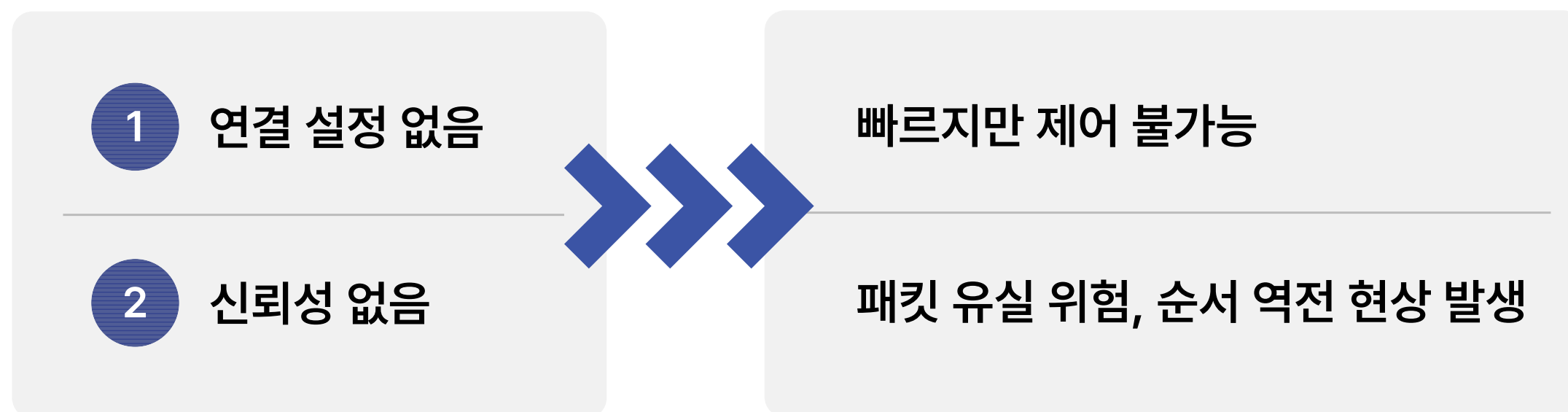
2022301043

이름

손유나

UDP 프로토콜의 한계점

핵심 구현 목표



💡 TCP의 신뢰성 전송 매커니즘
을 UDP 위에 직접 구현

도입 기술 키워드: Sequence Number, ACK, Timeout, Retransmission, Sliding Window

전체 시스템 구조 (System Architecture)

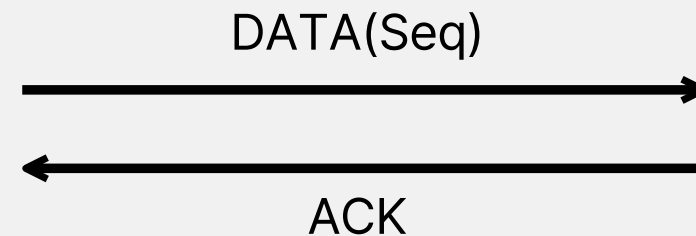
System Flow Diagram

[클라이언트 (Sender)]

- 파일 분할 (DATA_SIZE)
- 슬라이딩 윈도우 제어
- 타임아웃/재전송 처리

[서버 (Receiver)]

- 순서 검증 (expected_seq)
- 파일 스트림 순차 기록
- 누적 ACK 전송 (Ack_seq)



역할 분담

Sender(client): 데이터 패킷화, 윈도우 슬라이딩 제어, 유실 복구 주도

Receiver(serve): expected_seq 기준 엄격한 순서 검증, 파일 재조립 및 누적 ACK 응답

패킷 구조 설계 (Packet Format)

사용자 정의 패킷 구조체

TYPE	SIZE	DESCRIPTION
type	int (4byte)	패킷 종류 (TYPE_DATA, TYPE_ACK, TYPE_FIN)
seq	int (4byte)	패킷의 고유 시퀀스 번호 (순서 제어 및 ACK 매칭)
size	int (4byte)	실제 data 배열에 담긴 순수 데이터의 바이트 크기
data	DATA_SIZE (1024 byte)	실제 파일 조각 (바이너리 데이터)

type	seq	size	data
------	-----	------	------

CHAPTER 03

Stop-and-Wait ARQ 설계



⚠ Timeout 발생 시 동작



핵심 특징

- ✓ Sequence Number 사용
- ✓ ACK 수신 후 다음 패킷 전송
- ✓ Timeout 발생 시 재전송
- ✓ 중복 패킷 감지 및 무시

동작 요약

- 1 DATA(n) 전송
- 2 ACK(n) 수신 대기
- 3 Timeout 발생 시 DATA(n) 재전송
- 4 ACK(n) 수신 시 다음 패킷 전송

범례

- DATA 패킷 흐름
- - -> ACK 패킷 흐름

중복 패킷 처리 및 바이너리 예외 설계

ACK 유실 상황 발생 시 예외 처리

Receiver가 패킷 수신 및 저장 완료

회신한 ACK가 네트워크에서 유실

Sender는 타임아웃 후 동일 패킷 재전송

중복 패킷(Duplicate) 발생

해결책 (expected_seq 검증 메커니즘)

```
if (p_seq == expected_seq) {
```

순서가 딱 맞아떨어지는 패킷만 승인

```
} else {
```

순서가 어긋난 패킷이 오면 버리고,
"마지막으로 성공했던 ACK"를 재전송해
누적 ACK 유도

```
}
```

Sliding Window 도입 배경

Stop-and-Wait의 근본적 병목 원인

$$Throughput = Packet_{size} / RTT$$

Round Trip Time(왕복 시간)이 길어질수록 무의미한 대기 시간 급증



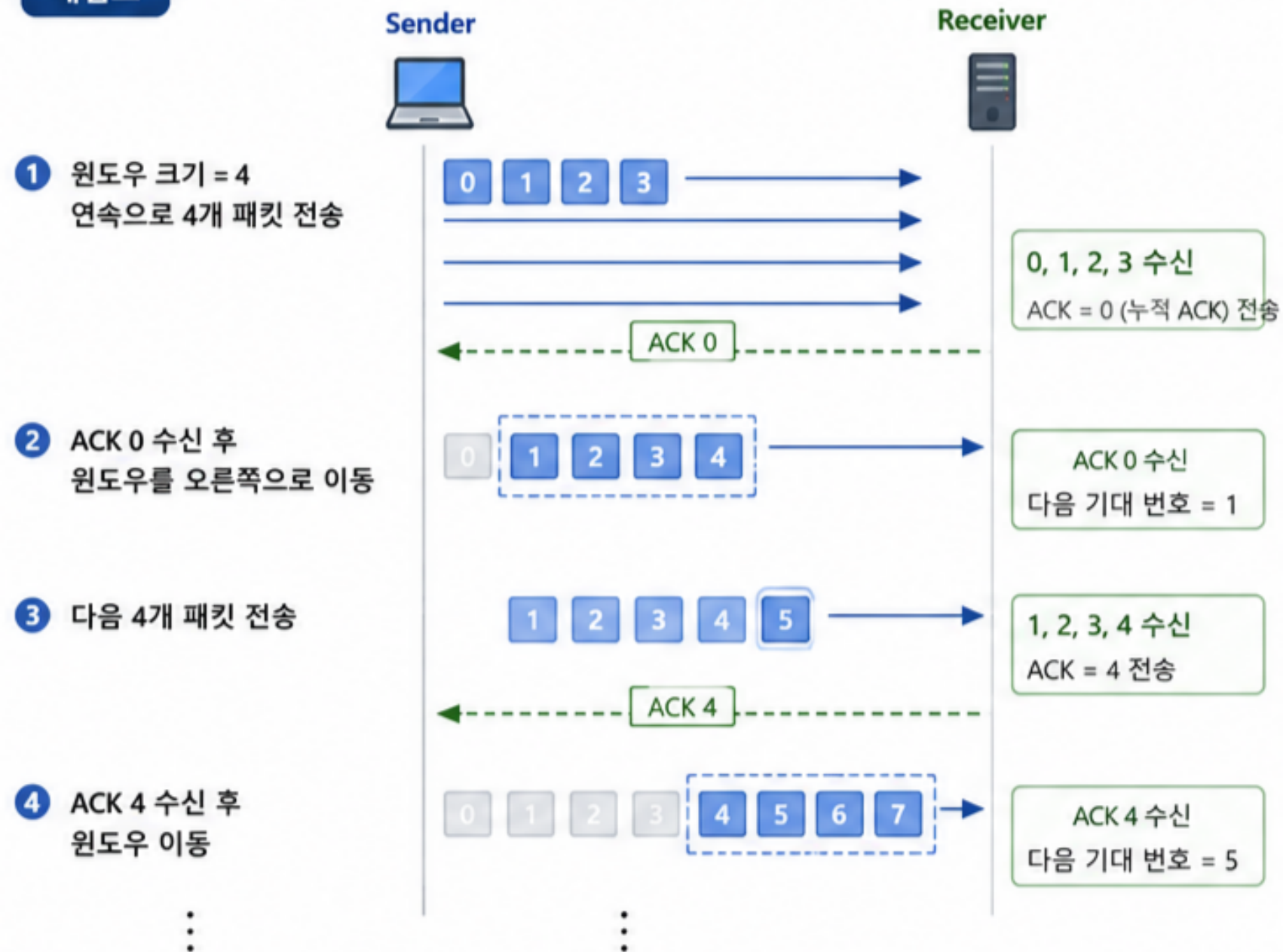
해결책

ACK 수신 여부와 관계없이, 승인받은 대역폭(WINDOW_SIZE)만큼 패킷을 멈춤 없이 연속으로 쏟아부어 **대기 시간을 최소화**

CHAPTER 06

Go-Back-N 설계 원리

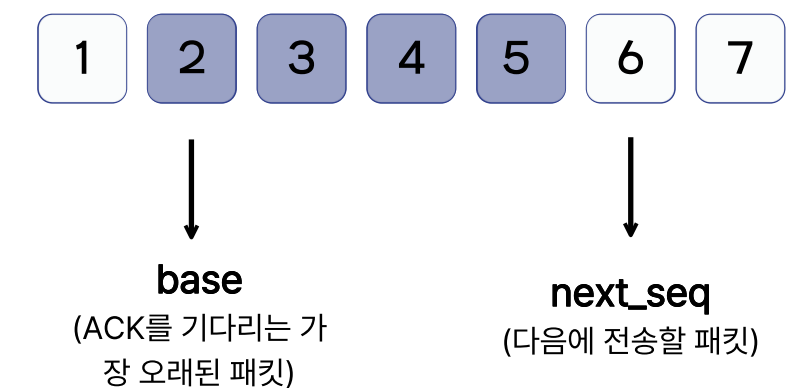
개념도



용어 설명

- base:** 윈도우의 가장 앞쪽에 있는 패킷 번호
(가장 오래된 미승인 패킷)
- next_seq:** 다음에 전송할 패킷
- WINDOW_SIZE:** 한번에 전송할 수 있는
최대 패킷 수
- 누적 ACK:** n번 ACK는 n번까지의 모든 패킷의
정상 수신되었음을 의미

윈도우 구조 예시 (윈도우 크기 = 4)

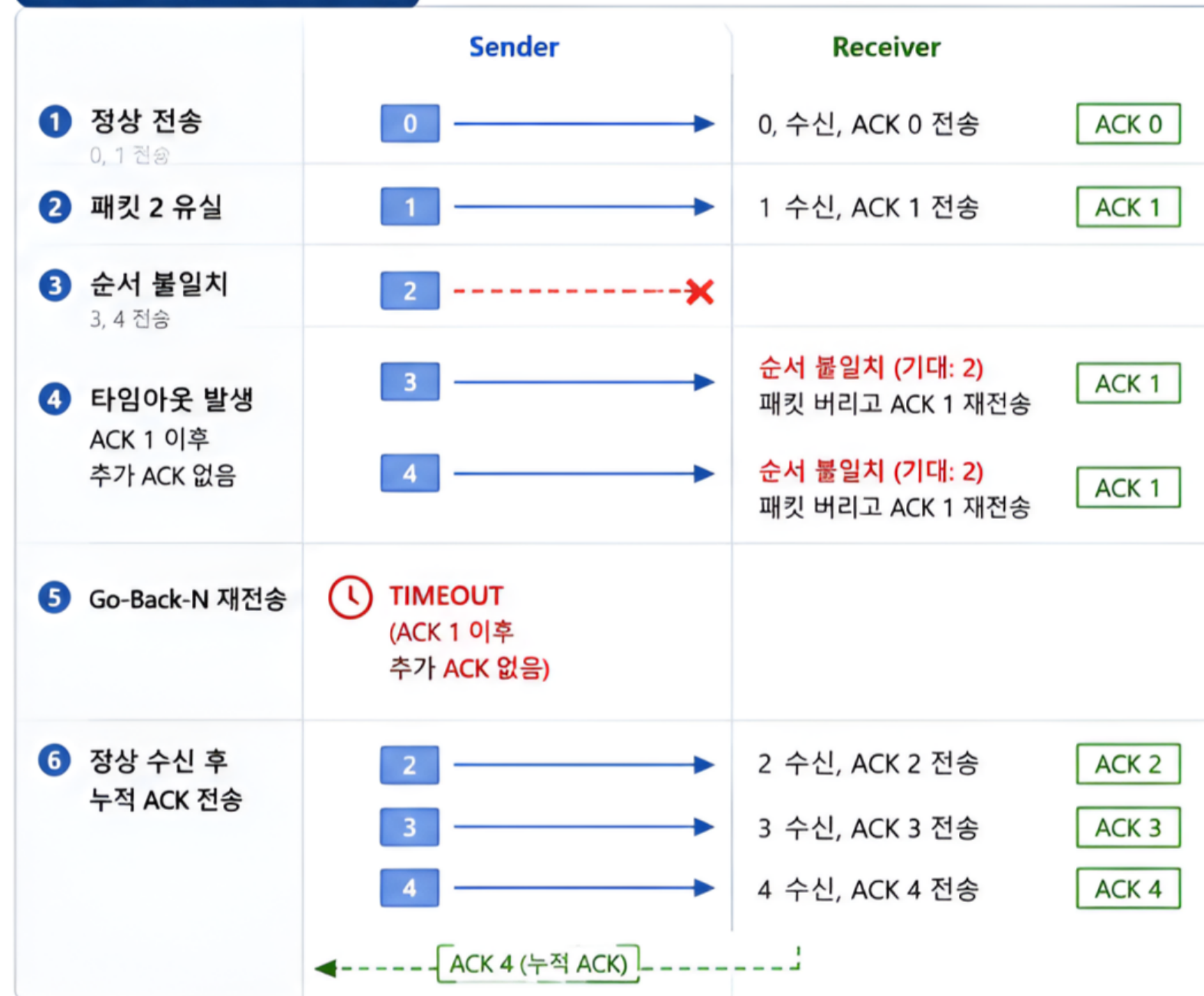


CHAPTER

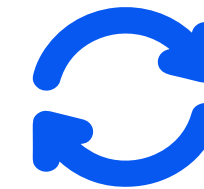
07

Go-Back-N 복구 과정 (Error Recovery)

동작 예시 (윈도우 크기 = 4)



핵심 특징



타임아웃 발생시
base 이후 모든 패킷을 재전송



순서가 맞지 않는 패킷을
버리고 마지막으로 받은
ACK를 재전송



구현이 단순하고
메모리 사용량이 적음



신뢰성 있는 전송 보장

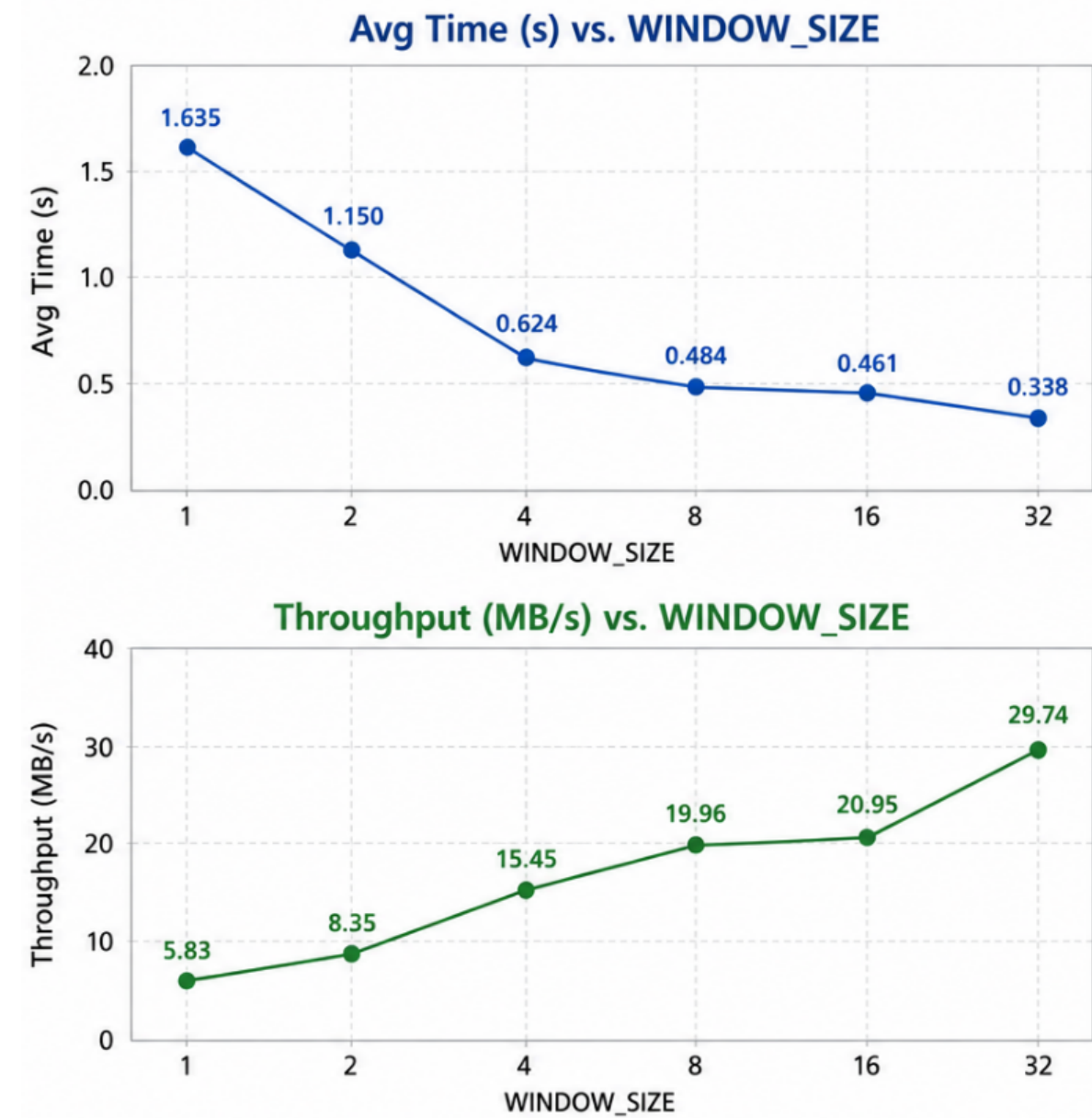
CHAPTER 08

성능 실험 환경 및 결과

테스트 환경

- OS : Ubuntu (WSL)
- 파일 크기 : 10MB
- DATA_SIZE : 1024 Byte
- 각 WINDOW_SIZE는 3회 반복 후 평균 측정
- 단위: Avg Time = 초(s), Throughput = MB/s

WINDOW_SIZE	Avg Time(s)	Throughput(MB/s)
1	1.635	5.83
2	1.150	8.35
4	0.624	15.45
8	0.484	19.96
16	0.461	20.95
32	0.338	29.74



결과

WINDOW_SIZE가 증가할수록 평균 전송 시간이 감소 하고 Throughput이 증가하는 경향을 알수 있습니다. WINDOW_SIZE가 16까지는 성능 향상이 뚜렷하며, 그 이상부터는 향상 폭이 다소 완만해집니다.



해석

윈도우 크기를 키울수록 한 번에 더 많은 패킷을 전송할수 있어 ACK대기 시간이 줄어들고, 전체 전송 효율(Throughput)이 향상됩니다.

CHAPTER
09

결론 및 향후 개선 방안

결과

구현 결과

Sequence Number, ACK, Timeout, Retransmission을 적용하여 패킷 유실 상황에서도 정상적인 파일 전송이 가능하도록 설계

Stop-and-Wait를 Go-Back-N Sliding Window 방식으로 확장하여 전송 효율을 향상

성능 결과

WINDOW_SIZE 증가에 따라 평균 전송 시간이 감소하고 Throughput이 증가하는 것을 확인

특히 Stop-and-Wait(WINDOW_SIZE=1) 대비 Go-Back-N 적용 시 전송 성능이 약 3배 이상 향상

배운점

Stop-and-Wait와 Go-Back-N을 직접 구현하면서 흐름 제어와 오류 제어의 동작 원리를 이해할 수 있었다.
TCP가 내부적으로 수행하는 기능들을 직접 구현해 보며 전송 계층 프로토콜의 동작 과정을 구체적으로 이해할 수 있었다.

향후 개선 방향

GBN은 높은 유실률 환경에서 불필요한 중복 전송 비용이 과다하게 발생함

향후 유실된 특정 패킷만 선택적으로 복구하는 Selective Repeat(SR) 프로토콜이나 TCP의 Fast Retransmit(빠른 재전송) 기법을 적용한다면 유실 환경에서도 대역폭을 방어할 수 있을 것으로 기대됨



2022301043



손유나

THANK YOU!